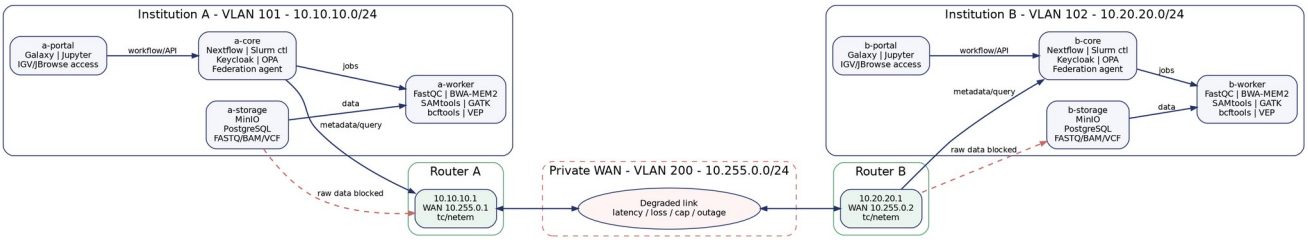
BUILDING A FEDERATED SOVEREIGN GENOMICS CLOUD ON A SINGLE PROXMOX NODE

Step-by-step implementation, DDIL testing, and reproducibility guide



Purpose: implement the proposed federation as two independent institutional clouds hosted as isolated virtual networks on one Proxmox node, while keeping raw genomic data local and testing what happens when the inter-site WAN is slow, lossy, bandwidth-limited, or unavailable.

This guide is designed for a research/lab environment. Do not place real identifiable human genomic data in this lab until institutional ethics, access-control, encryption, retention, and data-governance requirements are approved and implemented.

1. What you are building

The experiment creates two logically separate institutions on the same physical Proxmox host. Each institution has its own private VLAN, local storage, local workflow controller, local genomic worker, and user interface. The only permitted path between institutions is through a small routed WAN. That WAN becomes the controlled failure point used for latency, packet-loss, bandwidth, and outage experiments.

Layer	Site A / Site B component	Purpose
Virtualization	Proxmox VE	Creates VMs, bridges, VLAN isolation, snapshots, and virtual disks.
Private networks	VLAN 101 and VLAN 102	Separates the two simulated institutions.
Experimental WAN	VLAN 200 + two Linux router VMs	Creates a measurable link where tc/netem can inject DDIL conditions.
Object storage	MinIO first; Ceph optional later	Stores FASTQ, BAM/CRAM, VCF, reference files, and pipeline artifacts locally.
Workflow	Nextflow	Defines and executes the genomics pipeline.
Scheduling	Slurm initially; Kubernetes optional	Places tasks on worker nodes.
Genomics tools	FastQC, BWA-MEM2, SAMtools, Picard, GATK, bcftools, VEP/SnpEff	Actually transform raw reads into aligned data, variants, annotations, and reports.
User interface	Galaxy/Jupyter; IGV/JBrowse for visualization	Lets researchers run workflows and inspect results.
Identity/policy	Keycloak + OPA	Controls who can access what and whether data may leave a site.
Federation	Rust federation agent/API	Shares metadata and authorized requests without centralizing raw genomes.
Observability	Prometheus + Grafana + Rust metrics	Measures latency, throughput, errors, synchronization lag, CPU/RAM, and bytes crossing the site boundary.

The mid-term requires a falsifiable hypothesis, fixed evaluation rubric, Rust-based reproducible evidence, production validation, and a single documented reproduction command that regenerates the reported figures and tables from a clean checkout. The design below is organized around those requirements.

2. Recommended hypothesis and experimental claim

Primary hypothesis

Under degraded or disconnected WAN conditions, a federated genomics architecture in which each institution keeps local genomic storage and local workflow execution will preserve local

workflow availability and data residency better than a centralized architecture, while adding only bounded coordination overhead when connectivity is healthy.

Primary sovereignty dimension: operational sovereignty. Secondary dimensions: technical sovereignty and data residency. The key falsifiable result is not merely that “the system is sovereign”; it is whether local genomic workflows continue during WAN failure and whether the measured number of raw genomic bytes crossing the institutional boundary remains zero unless an explicit export policy permits them.

3. Physical and logical topology

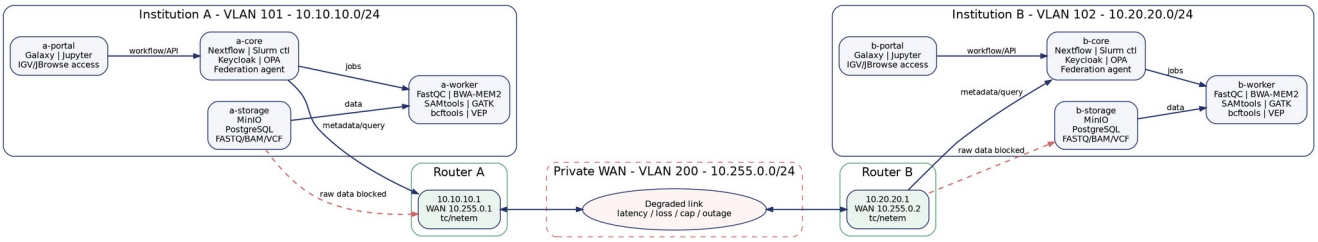


Figure 1. Laboratory topology. Both institutions run on the same Proxmox host, but their traffic is isolated by VLANs. Cross-site communication passes through Router A, the private WAN, and Router B. In a real deployment the two sites would be separate Proxmox clusters or separate physical infrastructures.

4. IP, VLAN, and VM plan

Network	VLAN	CIDR	Gateway	Role
Proxmox management	existing	your current management subnet	existing gateway	Do not modify unless necessary.
Institution A	101	10.10.10.0/24	10.10.10.1	Private Site A services and storage.
Institution B	102	10.20.20.0/24	10.20.20.1	Private Site B services and storage.
Experimental WAN	200	10.255.0.0/24	none	Router-A/Router-B link; tc/netem is applied here.

VM	vCPU	RAM	Disk	VLAN/IP	Initial role
router-a	2	2 GB	16 GB	101: 10.10.10.1; 200: 10.255.0.1	Routing, NAT for package install, tc/netem on WAN NIC.
router-b	2	2 GB	16 GB	102: 10.20.20.1; 200: 10.255.0.2	Routing, NAT, tc/netem.
a-core	4	8 GB	40 GB	101: 10.10.10.10	Nextflow, Slurm

					controller, Keycloak, OPA, federation agent.
a-storage	4	8 GB	200+ GB	101: 10.10.10.20	MinIO and PostgreSQL.
a-worker	8	16 GB	100+ GB	101: 10.10.10.30	Genomic processing tools.
a-portal	4	8 GB	60 GB	101: 10.10.10.40	Galaxy/Jupyter/ visualization access.
b-core	4	8 GB	40 GB	102: 10.20.20.10	Same as Site A.
b-storage	4	8 GB	200+ GB	102: 10.20.20.20	Same as Site A.
b-worker	8	16 GB	100+ GB	102: 10.20.20.30	Same as Site A.
b-portal	4	8 GB	60 GB	102: 10.20.20.40	Same as Site A.

Resource-constrained alternative: merge each site's core and portal functions into one VM and use one worker and one storage VM per site. The research logic stays the same; only isolation and scale change.

5. Phase A - prepare the Proxmox node

5.1 Verify the current host before changing networking

```
pveversion -v
ip -br addr
ip route
cat /etc/network/interfaces
pvesm status
qm list
```

Record the existing management bridge, management IP, default gateway, and physical NIC. In many installations the management bridge is vmbr0. Do not replace or renumber your existing management network for this lab.

5.2 Create one VLAN-aware private bridge

Use the Proxmox GUI: Node -> System -> Network -> Create -> Linux Bridge. Create vmbr1 with no physical bridge port. Enable VLAN aware. Do not assign an IP address to vmbr1.

```
auto vmbr1
iface vmbr1 inet manual
    bridge-ports none
    bridge-stp off
    bridge-fd 0
    bridge-vlan-aware yes
    bridge-vids 2-4094
```

Apply via the Proxmox GUI or, if you understand the host network and have console access, run:

```
ifreload -a
ip -d link show vmbr1
```

Because vmbr1 has no physical port, it is an internal virtual switch. VLAN tags on VM NICs create separate private broadcast domains on that switch.

5.3 Create VLAN-separated NICs on VMs

When attaching a VM NIC in Proxmox, set Bridge=vmbr1 and enter the VLAN tag. Site A VMs use tag 101; Site B VMs use tag 102; router WAN NICs use tag 200. Proxmox/Linux bridge switching keeps VLAN 101 and VLAN 102 isolated at Layer 2.

6. Phase B - create the two WAN router VMs

Use Ubuntu Server or Debian. Router A receives: one optional management/internet NIC on vmbr0, one Site-A NIC on vmbr1 tag 101, and one WAN NIC on vmbr1 tag 200. Router B is the same, using VLAN 102 and VLAN 200.

6.1 Identify the interfaces

```
ip -br link
ip -br addr
```

The examples below assume router-a interfaces ens18=management, ens19=Site A, ens20=WAN. Verify your actual names first.

6.2 Router A netplan example

```
network:
 version: 2
 ethernets:
  ens18:
   dhcp4: true
  ens19:
   addresses: [10.10.10.1/24]
  ens20:
   addresses: [10.255.0.1/24]
```

Router B uses 10.20.20.1/24 on its Site-B interface and 10.255.0.2/24 on its WAN interface.

```
sudo netplan generate
sudo netplan apply
ip -br addr
```

6.3 Enable forwarding and static routes

```
# Router A
echo 'net.ipv4.ip_forward=1' | sudo tee /etc/sysctl.d/99-genomics-router.conf
sudo sysctl --system
sudo ip route add 10.20.20.0/24 via 10.255.0.2

# Router B
echo 'net.ipv4.ip_forward=1' | sudo tee /etc/sysctl.d/99-genomics-router.conf
sudo sysctl --system
sudo ip route add 10.10.10.0/24 via 10.255.0.1
```

Make the routes persistent in netplan using routes: if desired. Verify with ip route.

6.4 Optional NAT for package installation

If the site VMs need internet access for initial package installation, NAT the local site subnet out through the router management NIC. Replace ens18 with the actual upstream interface.

```
sudo apt update && sudo apt install -y nftables
sudo tee /etc/nftables.conf >/dev/null <<'EOF'
flush ruleset
table inet filter {
 chain forward {
  type filter hook forward priority 0; policy drop;
  ct state established,related accept
  iifname "ens19" oifname "ens18" accept
  iifname "ens19" oifname "ens20" accept
  iifname "ens20" oifname "ens19" accept
 }
}
table ip nat {
 chain postrouting {
  type nat hook postrouting priority 100; policy accept;
  ip saddr 10.10.10.0/24 oifname "ens18" masquerade
 }
}
EOF
sudo systemctl enable --now nftables
```

Router B uses 10.20.20.0/24 in its NAT rule. Once the lab is built, restrict outbound internet access to what the experiment needs; the sovereignty experiment should not silently depend on external services.

7. Phase C - create and configure the site VMs

Create each VM in the Proxmox GUI or with qm. Ubuntu Server 24.04 LTS or a comparable supported Linux release is convenient. Use VirtIO SCSI disks and VirtIO network adapters. Assign the VLAN tag at the Proxmox NIC level.

7.1 Static network example for a-core

```
network:
 version: 2
 ethernets:
  ens18:
   addresses: [10.10.10.10/24]
   routes:
    - to: default
      via: 10.10.10.1
   nameservers:
    addresses: [1.1.1.1, 8.8.8.8]
```

For production or institutional use, replace public DNS with your approved local resolver. Site B uses gateway 10.20.20.1 and addresses in 10.20.20.0/24.

7.2 Establish SSH keys and hostnames

```
sudo hostnamectl set-hostname a-core
sudo apt update && sudo apt install -y openssh-server chrony curl jq git
sudo systemctl enable --now ssh chrony
```

Use SSH keys rather than shared passwords. Keep a simple /etc/hosts file or local DNS so site components can resolve a-core, a-storage, a-worker, and a-portal.

7.3 Basic local firewall policy

At minimum, allow SSH only from the administration subnet, allow MinIO only from local workflow/portal nodes, allow Slurm ports only inside the site, and expose Galaxy/Jupyter through the portal. Keep the inter-site WAN limited to federation API, monitoring, and other explicitly documented flows.

8. Phase D - deploy local genomic object storage

For the mid-term, MinIO is the easiest self-hosted object store to demonstrate. Use Ceph later if you want multi-node storage resilience. MinIO is for genomic objects; Proxmox storage/Ceph/ZFS continues to hold VM disks.

8.1 Docker installation on a-storage

```
sudo apt update
sudo apt install -y docker.io docker-compose-v2
sudo systemctl enable --now docker
sudo usermod -aG docker $USER
```

Log out/in after adding the docker group.

8.2 MinIO compose file

```
services:
 minio:
  image: quay.io/minio/minio:<PINNED_VERSION>
  command: server /data --console-address ':9001'
  restart: unless-stopped
  environment:
   MINIO_ROOT_USER: ${MINIO_ROOT_USER}
   MINIO_ROOT_PASSWORD: ${MINIO_ROOT_PASSWORD}
  ports:
   - '9000:9000'
   - '9001:9001'
  volumes:
   - /srv/minio:/data
```

Pin the exact image version or digest in your repository. Store credentials in an .env file excluded from Git. In a real deployment, terminate TLS and do not expose the MinIO administrative console to untrusted networks.

8.3 Create data buckets

```
mc alias set sitea http://10.10.10.20:9000 "$MINIO_ROOT_USER" "$MINIO_ROOT_PASSWORD"
mc mb sitea/raw
mc mb sitea/intermediate
mc mb sitea/results
mc mb sitea/reference
mc mb sitea/workflow-logs
```

Bucket	Typical objects	Sensitivity / policy
raw	FASTQ.gz, instrument exports	Highest sensitivity; no cross-site replication by default.
intermediate	SAM/BAM/CRAM, indexes	Still sensitive/identifiable; remains local.
results	VCF/gVCF, annotated VCF, QC reports	Controlled output; only approved derived results may leave.
reference	GRCh38/other reference data	Non-subject reference data; cache locally to survive WAN failure.
workflow-logs	Nextflow reports, provenance, checksums	Auditable operational evidence.

9. Phase E - install the genomics processing stack

The exact tools can vary by study. For the mid-term, use a small, public, non-sensitive dataset and a compact pipeline so that network behavior rather than biological scale dominates the experiment.

Stage	Software	Input	Output	What it does
QC	FastQC + MultiQC	FASTQ	HTML/ZIP QC reports	Checks read quality and summarizes multiple samples.
Trim (optional)	fastp	FASTQ	cleaned FASTQ	Removes adapters/low-quality reads.
Alignment	BWA-MEM2	FASTQ + reference	SAM/BAM	Maps reads to a reference genome.
BAM processing	SAMtools + Picard	SAM/BAM	sorted/indexed BAM	Sorts, indexes, and marks duplicates.
Variant calling	GATK	BAM/CRAM	gVCF/VCF	Identifies candidate genomic variants.
Filtering	bcftools	VCF	filtered VCF	Filters and manipulates variants.
Annotation	VEP or SnpEff	VCF	annotated VCF	Adds gene/consequence information.
Visualization	IGV/JBrowse;	BAM/VCF/results	interactive views/plots	Displays reads,

	Jupyter/R			variants, and statistical summaries.

Use containers for tools when possible so tool versions are pinned. Store the container image digests or exact tags in the repository.

10. Phase F - Nextflow and Slurm

OpenStack or Proxmox creates infrastructure; Slurm chooses where a task runs; Nextflow expresses the workflow; BWA/GATK/SAMtools actually process the genomic data.

```
Proxmox VM -> Linux -> Slurm worker -> Nextflow task -> BWA/GATK/SAMtools -> MinIO object
```

10.1 Minimal Slurm topology

- a-core: slurmctld controller.
- a-worker: slurmd worker.
- b-core and b-worker mirror the same pattern.
- Do not build a Slurm cluster spanning both institutions for the main hypothesis; each site must retain local execution during WAN failure.

10.2 Nextflow configuration concept

```
process.executor = 'slurm'
process.queue = 'genomics'
workDir = '/work'

profiles {
  sitea { params.site='A' }
  siteb { params.site='B' }
}
```

Nextflow should read raw objects from the local site, create work files on the local worker, and write final artifacts back to the local object store. The workflow must not assume that Site B is reachable.

11. Phase G - user-facing interface and visualization

Galaxy is useful for a browser-based scientific interface; JupyterLab is useful for analysis notebooks; IGV/JBrowse is useful for visual inspection of reads and variants. Keep infrastructure monitoring separate from genomic visualization: Grafana shows the cloud/experiment; IGV/JBrowse/Galaxy/Jupyter show the biological data and analysis.

Interface	User sees	Backend interaction
Galaxy	datasets, tools, workflows, history, reports	Submits local workflows; can call Slurm/containers.
JupyterLab	Python/R notebooks, plots, tables	Queries processed local data and produces analysis figures.

IGV/JBrowse	chromosome, reference, aligned reads, variants	Loads approved BAM/CRAM/VCF; avoid public exposure of sensitive objects.
Grafana	latency, loss, throughput, CPU/RAM, job state, sync lag	Prometheus/experiment metrics only; not a genomic browser.

12. Phase H - identity and policy

Deploy Keycloak locally at each site, or integrate a local identity provider if one already exists. The key requirement is that Site A users can still authenticate to Site A during WAN failure. Do not make local login dependent on a remote federation service.

OPA can enforce machine-checkable rules such as “FASTQ and BAM objects may not cross the institutional boundary” while allowing approved metadata or aggregate results.

```
# Illustrative OPA policy idea
package genomics.export

default allow = false

allow if {
  input.data_class == "metadata"
  input.user_role in {"researcher", "data_steward"}
}

allow if {
  input.data_class == "derived_result"
  input.approved == true
}
```

The exact policy must match your governance/ethics rules. The experiment should log policy decisions so the data-residency claim can be audited instead of merely asserted.

13. Phase I - build the Rust federation agent

Each site runs one small federation agent. It exposes metadata/capabilities and accepts authorized analysis requests. It should not expose a generic raw-object download endpoint.

Endpoint	Purpose	Raw genomic data?
GET /health	Local liveness/readiness	No
GET /capabilities	Which workflows the site can execute	No
GET /datasets	Approved metadata catalogue	No
GET /datasets/{id}	Metadata for one dataset	No
POST /analysis-request	Submit an authorized remote analysis request	Request only; processing happens locally

GET /jobs/{id}	Job state and approved result reference	No raw object by default
GET /metrics	Experiment/Prometheus metrics	No

Recommended Rust crates: tokio for async execution, tonic if you prefer gRPC, axum/actix-web for HTTP if simpler, serde for structured messages, tracing/OpenTelemetry for observability, and hdrhistogram for latency percentiles. The course specifically expects a Rust experimental harness and percentile-aware measurement.

14. Phase J - DDIL and WAN-failure testing

Apply network degradation to the WAN-facing interface of Router A and/or Router B, not to the internal site VLAN. This preserves local operation while degrading only federation traffic.

14.1 Baseline

```
sudo tc qdisc show dev ens20
ping -c 10 10.255.0.2
iperf3 -c 10.255.0.2
```

14.2 Inject latency

```
sudo tc qdisc replace dev ens20 root netem delay 100ms
```

14.3 Inject latency + packet loss

```
sudo tc qdisc replace dev ens20 root netem delay 150ms loss 5%
```

14.4 Bandwidth cap

```
sudo tc qdisc replace dev ens20 root tbf rate 2mbit burst 64kbit latency 400ms
```

14.5 Complete partition

```
sudo tc qdisc replace dev ens20 root netem loss 100%
```

14.6 Restore

```
sudo tc qdisc del dev ens20 root || true
```

For symmetric degradation, apply the same condition to both routers' WAN-facing interfaces. Record exactly which direction(s) were shaped and keep that constant across repeated runs.

15. Experimental matrix

Scenario	Added latency	Loss	Bandwidth	Expected local workflow	Expected federation
S0 Healthy	0 ms	0%	unrestricted	available	available
S1 Mild latency	50 ms	0%	unrestricted	available	slightly slower

S2 High latency	150 ms	0%	unrestricted	available	slower
S3 Severe latency	300 ms	0%	unrestricted	available	much slower
S4 Loss	100 ms	2%	unrestricted	available	retries/errors possible
S5 Severe loss	100 ms	10%	unrestricted	available	strong degradation
S6 Cap	baseline	0%	10 Mbps	available	transfer-limited
S7 Severe cap	baseline	0%	2 Mbps	available	strongly limited
S8 Partition	n/a	100%	0	MUST remain available	unavailable/ queued/cached

Run each configuration at least 30 independent repetitions for the headline measurements. Record raw observations, then report mean plus 95% confidence intervals and tail latency (p50/p95/p99 where relevant).

16. What to measure

Metric	How to measure	Why it matters
Local workflow success rate	completed/started jobs at each site	Direct test of continued operation during WAN failure.
Workflow completion time	Nextflow/Slurm timestamps	Shows whether local compute is insulated from WAN conditions.
Federation p50/p95/p99 latency	Rust hdrhistogram around API calls	Quantifies coordination overhead and degradation.
Federation throughput	operations/sec or requests/sec	Shows useful work under constrained links.
Recovery time	time from WAN restoration to normal federation	Fault-tolerance evidence.
Metadata sync lag	difference between local update and remote visibility	Measures reconciliation behavior.
Raw genomic bytes crossing boundary	router counters, flow logs, app instrumentation	Critical residency/sovereignty metric; target zero unless explicitly authorized.
CPU/RAM/disk/network	cgroups/system metrics/Prometheus	Resource footprint.
External dependency count	manual + automated dependency audit	Operational sovereignty.

17. Prometheus and Grafana

Collect metrics locally at each site so the monitoring plane itself does not become a dependency. If you also use a central observer, treat it as an observer only; loss of the observer must not stop workflows.

- Export Rust federation metrics such as request latency, request count, failures, queued operations, and sync lag.
- Export node metrics for CPU/RAM/disk/network.
- Export application metrics for Nextflow/Slurm where practical.
- Plot the WAN condition and system behavior on the same timeline so a fault can be correlated with workflow and federation behavior.

18. A clean Git repository layout

```
federated-genomics-cloud/
├── README.md
├── Makefile
├── Cargo.toml
├── .gitignore
├── infra/
│   ├── proxmox/            # documented VM/VLAN plan; optional Terraform later
│   ├── ansible/            # host configuration
│   └── inventories/lab.ini
├── federation/
│   └── src/                # Rust federation agent
├── harness/
│   └── src/                # Rust experiment driver
├── workflows/
│   ├── main.nf
│   └── nextflow.config
├── policies/
│   └── export.rego
├── docker/
│   ├── compose-site.yml
│   └── analysis.Dockerfile
├── experiments/
│   ├── scenarios.yaml
│   └── run_matrix.sh
├── data/
│   └── raw/                # committed experiment CSV/JSON, not sensitive genomes
├── analysis/
│   └── reproduce.py
└── results/
    ├── figures/
    └── tables/
```

Never commit real human genomic files, credentials, tokens, or private keys. Commit only non-sensitive experimental measurements and small public/test fixtures if licensing permits.

19. The required one-command reproduction workflow

This requirement means that an examiner should clone the repository, run one command, and regenerate every figure and table in your report from the committed raw experimental measurements. It does not mean the examiner must rebuild your entire Proxmox lab every time. Separate infrastructure deployment from analytical reproducibility.

Recommended command

```
make reproduce
```

19.1 Makefile

```
REPRO_IMAGE := federated-genomics-repro:1.0

.PHONY: reproduce clean test experiment

reproduce:
	docker build -f docker/analysis.Dockerfile -t $(REPRO_IMAGE) .
	docker run --rm \
	  -v "$(PWD):/work" -w /work \
	  $(REPRO_IMAGE) bash -lc './experiments/reproduce.sh'

clean:
	rm -rf results/figures results/tables
	mkdir -p results/figures results/tables

test:
	cargo test --workspace

experiment:
	cargo run --release -p harness -- \
	  --config experiments/scenarios.yaml \
	  --output data/raw
```

19.2 Pinned analysis container

```
FROM python:3.12.7-slim@sha256:<PIN_THE_REAL_DIGEST>
RUN pip install --no-cache-dir \
   pandas==2.2.3 \
   matplotlib==3.9.2 \
   scipy==1.14.1
WORKDIR /work
```

Replace the digest with the exact digest you use. The digest pins the base image; package versions pin the analysis stack.

19.3 reproduce.sh

```
#!/usr/bin/env bash
set -euo pipefail

rm -rf results/figures results/tables
mkdir -p results/figures results/tables

# Verify the repository software still builds/tests.
cargo test --workspace

# Regenerate every table and figure from committed raw measurements.
python analysis/reproduce.py \
 --input data/raw \
 --figures results/figures \
 --tables results/tables
```

```
# Fail if expected artifacts are missing.
test -s results/figures/latency_vs_condition.png
test -s results/figures/workflow_success.png
test -s results/tables/summary.csv
test -s results/tables/confidence_intervals.csv

echo 'Reproduction complete.'
```

19.4 Minimal reproduce.py responsibilities

- Load every raw CSV/JSON file under data/raw.
- Validate required columns and scenario names.
- Calculate mean, median, p50, p95, p99, standard deviation, and 95% confidence intervals where appropriate.
- Generate the exact charts used in the manuscript.
- Generate the exact CSV/Markdown/LaTeX tables used in the manuscript.
- Use deterministic sorting and fixed analysis rules so repeated runs produce the same values.
- Exit non-zero if data are missing or malformed.

Example conceptual CLI:

```
python analysis/reproduce.py --input data/raw --figures results/figures --tables results/tables
```

19.5 Clean-checkout verification

```
git clone https://example.org/your-team/federated-genomics-cloud.git
cd federated-genomics-cloud
git checkout <TAG_USED_FOR_SUBMISSION>
make clean
make reproduce
```

Your README should state the expected runtime, Docker requirement, hardware assumptions, and the list of files that should appear in results/figures and results/tables.

20. Rust experiment harness design

The harness should control the test, generate traffic, measure results, and write raw measurements. A simple model is:

```
scenario -> apply WAN condition -> warm-up -> run N requests/jobs -> record samples -> restore WAN -> write CSV
```

Raw column	Example	Meaning
run_id	S5-017	Unique repetition identifier.
scenario	loss_10pct	Experimental condition.
site	A	Origin site.
request_type	metadata_query	Operation under test.
latency_ms	483.2	Per-request latency.
success	true	Whether request/job completed.

workflow_seconds	91.4	Local workflow duration.
bytes_cross_site	0	Measured cross-boundary data volume.
timestamp_utc	...	Event time for correlation.

Keep the raw data granular. Do not commit only averages: the reproduction command needs the individual observations to recompute uncertainty and tail latency.

21. Automating network conditions

Your Rust harness can SSH to router-a/router-b or call a small privileged local helper on each router. Keep privileged operations separate from the unprivileged measurement process.

```
# examples executed on router WAN interface
tc qdisc replace dev ens20 root netem delay 150ms loss 5%
# run experiment
tc qdisc del dev ens20 root || true
```

Always use a trap/finally cleanup so a failed experiment does not accidentally leave the WAN shaped for later scenarios.

22. Baseline comparisons

The course package names OpenStack private cloud, CRANE Cloud, lightweight Kubernetes, and a centralized single-node service as mandatory baselines. Proxmox can host your proposed system, but your report should still compare your architecture against the required baseline concepts. If you later receive OpenStack access, deploy the same application stack there so the application/workflow logic stays comparable while the infrastructure layer changes.

For the centralized baseline in the Proxmox lab, create one central genomics service and make Site A access it across the experimental WAN. During a partition, the client loses the central service. Compare this with the federated design where the local site continues processing.

23. Validation sequence for the live demo

1. Open Grafana and show healthy WAN metrics.
2. Upload or select a small public FASTQ at Site A.
3. Start the local pipeline and show Nextflow/Slurm dispatching the job.
4. Show FASTQ in Site A raw storage and verify no copy exists in Site B.
5. Run a cross-site metadata discovery request successfully.
6. Inject 150 ms latency and 5% loss; show increased federation latency while the local pipeline continues.
7. Inject a 100% WAN loss/partition.
8. Show Site A and Site B local portals, storage, authentication, and genomic workflows still functioning.

9. Show cross-site discovery failing/queueing/caching as designed.
10. Restore the WAN and measure reconciliation/recovery time.
11. Show the raw-genomic-bytes-crossing-boundary metric stayed at zero.
12. Run or display the clean-checkout command: make reproduce, and show regenerated figures/tables.

24. What counts as a successful result

Claim	Pass condition	Evidence
Local operation survives WAN loss	Local genomic job success remains near healthy baseline during S8	Nextflow/Slurm logs + harness CSV + confidence intervals.
Raw data remains local	0 raw FASTQ/BAM/CRAM bytes cross inter-site WAN unless explicitly approved	Router/app counters + OPA logs.
Federation degrades gracefully	Healthy -> latency/loss causes measurable latency/throughput change rather than hidden failure	p50/p95/p99 and success curves.
System recovers	Federation returns and metadata reconciles after link restoration	Recovery-time and sync-lag metrics.
Results are reproducible	Fresh clone + make reproduce regenerates all reported figures/tables	Tagged repository + pinned container + committed raw measurements.

25. Common mistakes to avoid

- Do not stretch one Proxmox cluster across unreliable real sites and then claim independent failure domains. For the lab, one host is an emulation convenience; production sites should be independent.
- Do not put Keycloak, MinIO, Nextflow, or Slurm only in a central site if your claim is that each institution can continue locally.
- Do not synchronize raw FASTQ/BAM automatically across sites if the contribution is data-local federation.
- Do not benchmark only the happy path; the strongest experiment is the WAN failure case.
- Do not report one run or only an average. Preserve raw samples and report uncertainty and tail latency.
- Do not make figures manually in Excel and then claim one-command reproducibility. The exact submission figures/tables must be produced by make reproduce.
- Do not commit credentials, private keys, or identifiable genomic data to the Git repository.
- Do not silently depend on internet-hosted container registries or identity services during the outage experiment; pre-pull/cache what is required or clearly count the dependency.

26. Suggested build order for the next working sessions

Order	Deliverable	Stop/go test
1	vmbr1 + VLAN 101/102/200	VMs on different VLANs cannot talk

		without routers.
2	router-a and router-b	A can route to B only through 10.255.0.0/24.
3	Site A/B basic VMs	All local hosts resolve/ping inside their own site.
4	MinIO both sites	Local upload/download works; cross-site raw export blocked.
5	FastQC/BWA/SAMtools/GATK test pipeline	Small public sample runs locally end-to-end.
6	Nextflow + Slurm	Workflow runs through scheduler on each site independently.
7	Galaxy/Jupyter	Researcher can launch/view local analysis through browser.
8	Rust federation agent	Metadata query works A<->B without raw data transfer.
9	OPA/Keycloak	Unauthorized/raw export request is denied and logged.
10	Prometheus/Grafana	WAN and application metrics visible.
11	Rust harness + scenarios	30-run scenario matrix writes raw CSV.
12	make reproduce	Fresh checkout recreates all figures and tables.
13	Real degraded-network trial	At least one non-emulated constrained condition is measured and compared.

27. Reference to the examination requirements

The implementation plan intentionally maps to the supplied MCS 7101 mid-term package: one falsifiable hypothesis and concept family; systematic baseline comparison; Rust-based experimental evidence; controlled latency/loss/bandwidth tests; at least one real degraded-network validation; confidence intervals and repeated runs; and a public/tagged reproducibility package with a one-command regeneration of figures and tables.

The supplied paper review also supports the architectural principle used here: institutionally controlled compute/storage, local data custody, workflow/container portability, and federation that moves authorized queries or computation toward governed data rather than centralizing raw genomic data.

28. Final checklist

- ☐ Proxmox private VLANs created and documented.
- ☐ Site A and Site B cannot bypass the experimental WAN.
- ☐ Each site can authenticate, store data, schedule jobs, and run the genomics pipeline locally.
- ☐ Raw genomic objects are local by default.

- ☐ Federation API exchanges only approved metadata/requests/results.
- ☐ OPA decisions are logged.
- ☐ WAN scenarios are scripted and reversible.
- ☐ At least 30 repetitions are captured for headline configurations.
- ☐ Raw results are committed in a non-sensitive form.
- ☐ Figures/tables are generated only from committed raw results.
- ☐ `make reproduce` works from a clean clone.
- ☐ One real degraded-network trial is documented.
- ☐ Live demo shows a real fault and recovery.